

1. (10 points) True/False Questions. Circle T for true and F for false. If further clarification is needed, feel free to write down your own assumption and clarification.
- (a) (1 point) (T/F) Reflex agents are *always* optimal since they are not planning agents.
 - (b) (1 point) (T/F) Planning agents make decisions based on (hypothesized) consequences of actions.
 - (c) (1 point) (T/F) For efficient planning, a search state *must* keep every last detail of the environment as the world state.
 - (d) (1 point) (T/F) In a search tree, a unique state appears only once and this leads to a compact representation of the world.
 - (e) (1 point) (T/F) A DFS search is *not* optimal in general since it finds the “left-most solution”.
 - (f) (1 point) (T/F) Solutions to relaxed problems can be used for admissible heuristics.
 - (g) (1 point) (T/F) In the A^* algorithm, admissible (optimistic) heuristics slow down bad plans but never outweigh true costs.
 - (h) (1 point) (T/F) If a heuristic is not admissible then the heuristic is not consistent.
 - (i) (1 point) (T/F) Given two admissible heuristics h_1 and h_2 , new heuristics can be obtained by both min and max, *i.e.*, $h_{\min}(n) := \min(h_1(n), h_2(n)), \forall n$, and $h_{\max}(n) := \max(h_1(n), h_2(n)), \forall n$, where n is a state. $h_{\max}$ and $h_{\min}$ are not admissible and $h_{\max}$ dominates $h_{\min}$.
 - (j) (1 point) (T/F) A^* algorithm stops when a goal state is dequeued.

2. (10 points) True/False Questions. Circle T for true and F for false. If further clarification is needed, feel free to write down your own assumption and clarification.
- (a) (1 point) (T/F) In zero-sum games, agents have opposite utilities. It can be viewed as a single value that one maximizes and the other minimizes. It is an adversarial and pure competition.
 - (b) (1 point) (T/F) Adversarial search via minimax search computes the best achievable utility against a rational adversary at each node.
 - (c) (1 point) (T/F) In minimax search, evaluation functions return an approximate minimax values at non-terminal nodes for depth-limited search.
 - (d) (1 point) (T/F) Alpha-beta pruning improves the efficiency of minimax search but it changes the minimax values on intermediate nodes and the root node.
 - (e) (1 point) (T/F) In Alpha-beta pruning, a good child ordering improves the effectiveness of pruning.
 - (f) (1 point) (T/F) Some games have inherent randomness such as rolling dice, or unpredictable opponents, and action failures. In this case, values should reflect average-case outcomes and expectimax can be used.
 - (g) (1 point) (T/F) Minimax search assumes the opponent is rational (optimal). Even if the opponent chooses random actions, minimax always outperforms expectimax since minimax plans against smarter agents.
 - (h) (1 point) (T/F) The actions chosen by minimax search and expectimax are insensitive to scaling with a positive factor, *i.e.*, $v_{\text{new}} = \alpha v_{\text{old}}$, where $\alpha > 0$.
 - (i) (1 point) (T/F) Utilities are functions that map outcomes (states of the world) to real numbers that describe agent's preferences
 - (j) (1 point) (T/F) According to 'Maximum expected utility (MEU) principle', rational agents choose the action that maximizes expected utility. Also, an agent can be entirely rational (consistent with MEU) without ever representing or manipulating utilities and probabilities.

3. (10 points) **Uninformed Search.**

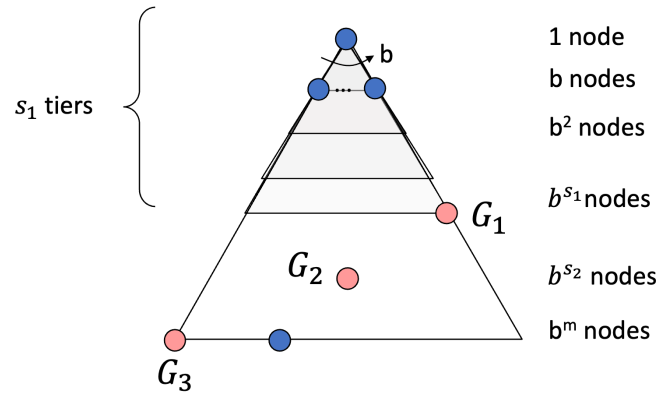


Figure 1: Search Tree. G_1 , G_2 , G_3 denote the goal states with at tier s_1 , s_2 , m , respectively.

- (a) (2 points) Assume that tree search algorithms consider left nodes first. Which of goal nodes among G_1 , G_2 , G_3 will be returned by DFS and BFS, respectively.
 - (a) DFS:
 - (b) BFS:
- (b) (4 points) Given a search tree with $b = 9$, $m = 10$, $s_1 = 5$, $s_2 = 7$, compute the number of nodes to expand until a goal node is found. Among DFS, and BFS, which algorithm needs less computation to find a solution?
- (c) (2 points) Assuming that the cost of every action is '1', compute the total cost of each plan/goal. Also, discuss the optimality of solutions obtained from DFS, BFS.
 - (a) G_1 :
 - (b) G_2 :
 - (c) G_3 :
 - (d) Optimality of DFS, and BFS:
- (d) (2 points) Compute the maximum size of fringe that is required to find a solution by DFS, and BFS. In Fig. 1, which algorithm requires smaller memory
 - (a) Maximum number of nodes in the fringe (DFS):
 - (b) Maximum number of nodes in the fringe (BFS):

4. (10 points) **Informed Search: A^* search and heuristics.**

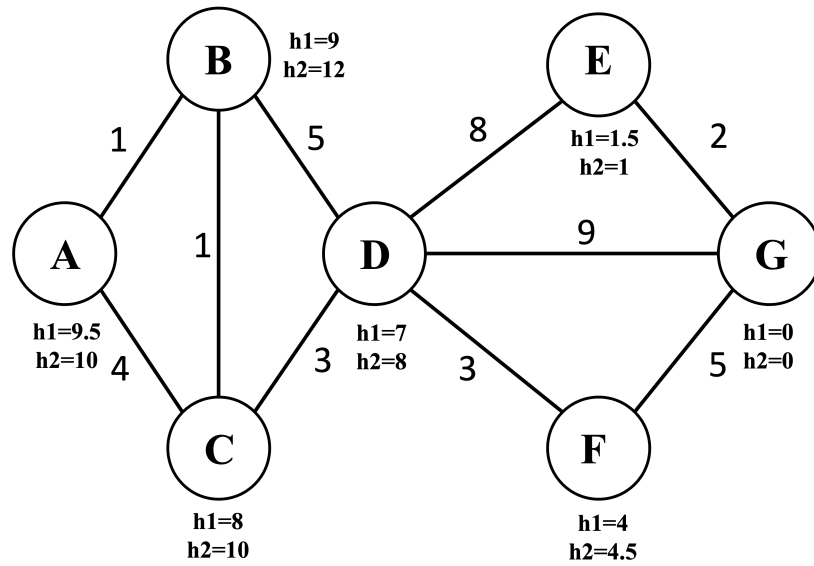


Figure 2: State Space Graph. h_1 and h_2 are two heuristic functions. A and G are the start and goal states. The numbers on edges are costs and edges can be traversed in both directions.

- (a) (5 points) Check all the possible solutions that can be returned by each of the following algorithms (use graph search).

Algorithm	A-B-D-G	A-C-D-G	A-B-C-D-F-G
Depth First Search			
Breadth First Search			
Uniform Cost Search			
Greedy Search w/ h_1			
Greedy Search w/ h_2			
A^* with h_1			

- (b) (3 points) Explain the consistency and show that h_2 is not a consistent heuristic.

- (c) (2 points) For the pathing problem that aims at finding the shortest path from a start state S to a goal state G , the Euclidean distance from each state n to

the goal state G is an admissible heuristic. Explain why the Euclidean distance is always smaller than or equal to the actual cost (*i.e.* length of the shortest path).

5. (15 points) **Minimax search, alpha-beta pruning, uncertainty and utilities.**

- (a) (1 point) Perform minimax search without any pruning. Compute values at non-terminal nodes ($A, \dots, F$).

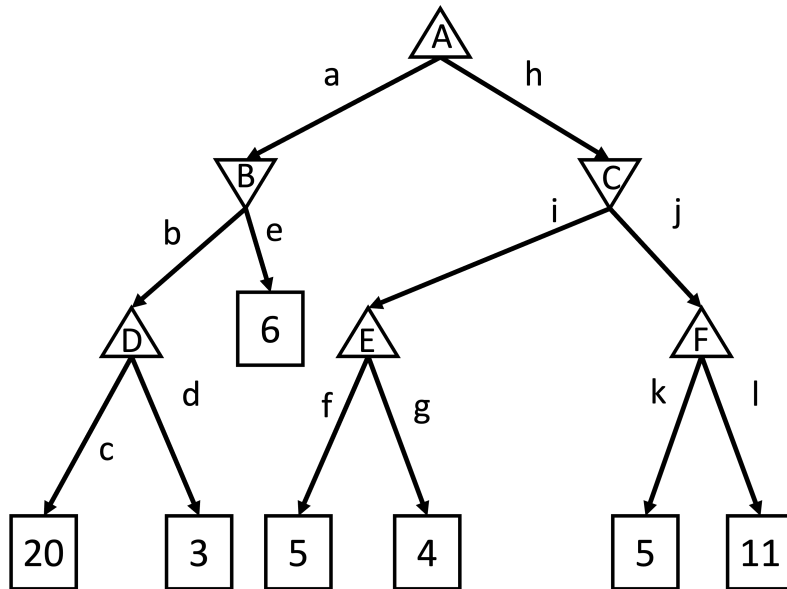


Figure 3: Minimax search tree.

- (b) (5 points) Perform a more efficient search called alpha-beta pruning.

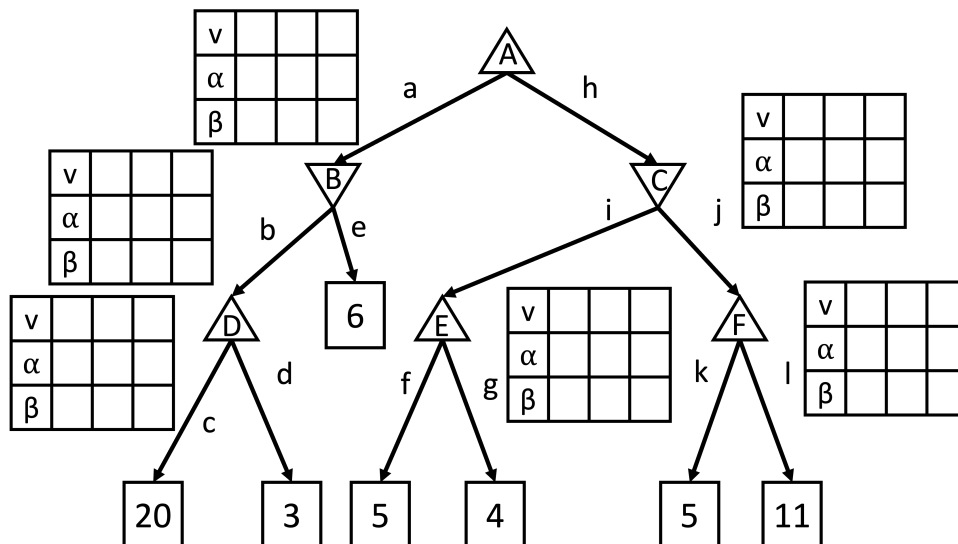


Figure 4: Alpha-beta pruning search tree.

- (c) (2 points) Discuss any reordering of edges that leads to more effective alpha-beta pruning.

Answer: If edge () and () are reordered, then edge () will be pruned.

- (d) (2 points) Alpha-beta pruning is one way to improve the efficiency of the minimax algorithm. But in the worst case, it fails to prune a node and still requires a large computational resource. In realistic games with a huge depth and possible actions, how can we find a solution? (Hint. Depth, approximation, terminal utilities, non-terminals)

- (e) (5 points) Rational preferences can be defined with the axioms of rationality.

$$\text{Orderability: } (A \succ B) \vee (B \succ A) \vee (A \sim B) \quad (1)$$

$$\text{Transitivity: } (A \succ B) \wedge (B \succ C) \Rightarrow (A \succ C) \quad (2)$$

$$\text{Continuity: } A \succ B \succ C \Rightarrow \exists p \ [p, A; 1 - p, C] \sim B \quad (3)$$

$$\text{Substitutability: } A \sim B \Rightarrow [p, A; 1 - p, C] \sim [p, B; 1 - p, C] \quad (4)$$

$$\text{Monotonicity: } A \succ B \Rightarrow (p \geq q \Leftrightarrow [p, A; 1 - p, B] \succeq [q, A; 1 - q, B]) \quad (5)$$

The rational preferences imply that behaviours are describable as the maximization of expected utility (MEU). Let us define the utility values as $U(A) = a, U(B) = b, U(C) = c$, where a, b, c are real values. Answer the following questions. (Hint. You may want to rewrite down the axiom with utility function values using the following additional notations: $=, >, U(\cdot)$).

1. (2 points) Continuity: Find p . (Hint. $U([p, A; 1 - p, C]) = U(B)$).

2. (3 points) Monotonicity ($\Rightarrow$ only): Show that if $(a > b) \wedge (p \geq q)$, then $U([p, A; 1 - p, B]) \geq U([q, A; 1 - q, B])$.